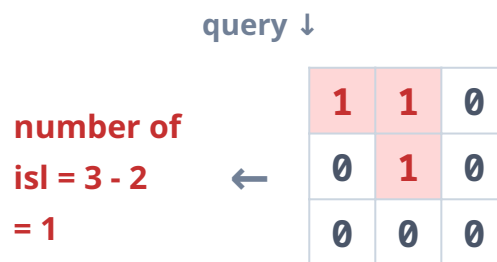


LC 305: Number of Islands II

Authored by: Arpit Raj, LNMIIT Jaipur • Date: August 4, 2026

Algorithm Explanation & Visualizations

the grid starts as



hence

S1 After a query whenever a new land appears we do **cnt++**

S2 check all 4 neighbours, if neighbour is a land and belongs to a different component we merge both and do **cnt --** , $\therefore$ 2 new islands become one

the question also has duplicate queries

hence

```
if (land[node] == 1) {  
    ans.push_back(count);  
    continue;  
}
```

**this concept and algorithm is called
DYNAMIC DSU**

C++ Implementation

```
class DisjointSet {  
public:  
    vector<int> parent;  
    vector<int> size;  
  
    DisjointSet(int n) {  
        parent.resize(n);  
        size.resize(n, 1);  
        for(int i = 0; i < n; i++) {  
            parent[i] = i;  
        }  
    }  
  
    int findParent(int node) {  
        if(parent[node] == node) {  
            return node;  
        }  
        return parent[node] = findParent(parent[node]);  
    }  
  
    void unionBySize(int u, int v) {  
        int pu = findParent(u);  
        int pv = findParent(v);  
  
        if(pu == pv) {  
            return;  
        }  
  
        if(size[pu] < size[pv]) {
```

```

        swap(pu, pv);
    }

    parent[pv] = pu;
    size[pu] += size[pv];
}

};

class Solution {
public:
    vector<int> numIslands2(int n, int m, vector<vector<int>>& positions) {
        DisjointSet ds(n * m);
        vector<int> ans;

        // initially sab water hain
        vector<int> land(n * m, 0);

        vector<pair<int, int>> dir = {
            {-1, 0},
            {0, -1},
            {1, 0},
            {0, 1}
        };

        int islands = 0;

        for(auto &it : positions) {
            int r = it[0];
            int c = it[1];
            int curr = r * m + c;

            // Agar pehle se land hai
            // to island count change nahi hoga
            if(land[curr]) {
                ans.push_back(islands);
                continue;
            }

            // water -> land
            // nayi island maan lo
            land[curr] = 1;
            islands++;

            for(auto &d : dir) {
                int nr = r + d.first;
                int nc = c + d.second;

                if(nr >= 0 && nc >= 0 && nr < n && nc < m) {
                    int next = nr * m + nc;

                    // sirf land neighbour se merge hoga
                    if(land[next]) {
                        int p1 = ds.findParent(curr);
                        int p2 = ds.findParent(next);

```

```
        // alag islands hain to merge kr denge and islands-- as 2 island m
        if(p1 != p2) {
            ds.unionBySize(curr, next);
            islands--;
        }
    }
}
ans.push_back(islands);
}
return ans;
}
};
```